`pi-game-trees.typ` — Extensive-Form Game Trees in CeTZ

This document demonstrates the `pi-game-trees.typ` macro library for drawing extensive-form game trees in Typst using CeTZ 0.5.2. The visual design follows the conventions of the *xgames* LaTeX package: each player's name, action labels, and payoffs are rendered in a dedicated colour for clarity. Unlike xgames, the syntax conforms strictly to Typst and CeTZ conventions.

Player colours:

Player 1 Player 2 Player 3 Player 4 Player 5 Nature

Example 1 — Entry Deterrence (Sequential, Perfect Information)

A **Challenger** decides whether to enter a market. If she enters, the **Incumbent** either *fights* or *accommodates*. The unique SPNE survives backward induction: the Challenger enters, the Incumbent accommodates, yielding payoffs (1, 1).

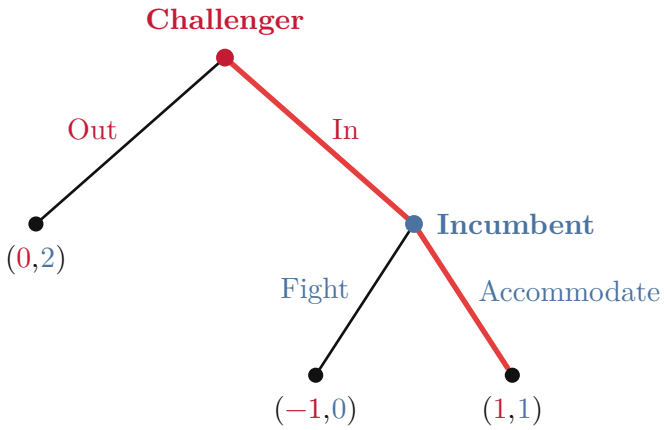

Figure 1: Entry Deterrence — the bold path is the SPNE.

The highlighted branches show the backward-induction equilibrium: **Incumbent** prefers to accommodate (payoff $1 > 0$), so **Challenger** enters.

Example 2 — Battle of the Sexes (Simultaneous Moves)

Player 1 moves first in the tree, but **Player 2** does not observe the move — hence the two **Player 2** nodes belong to the same **information set** (dashed line). This makes the game strategically equivalent to its normal form.

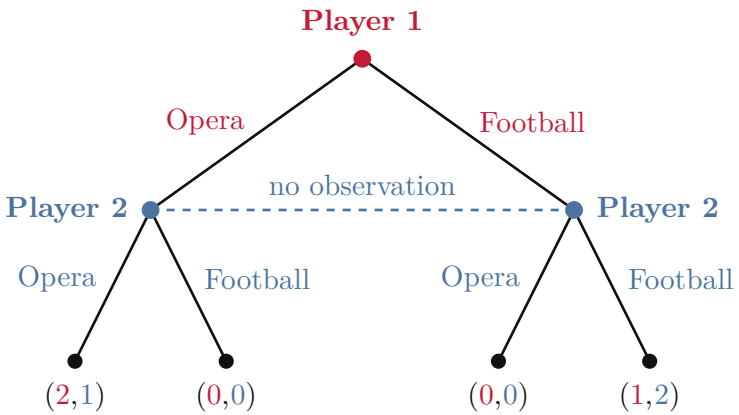

Figure 2: Battle of the Sexes — extensive form with information set. Two pure-strategy NE: (2, 1) and (1, 2).

Example 3 — Adverse Selection with Nature

Nature draws a buyer's valuation: *Low* (θ_L) with probability $\frac{1}{3}$ and *High* (θ_H) with probability $\frac{2}{3}$. Only the buyer observes her type; the seller offers a price p.

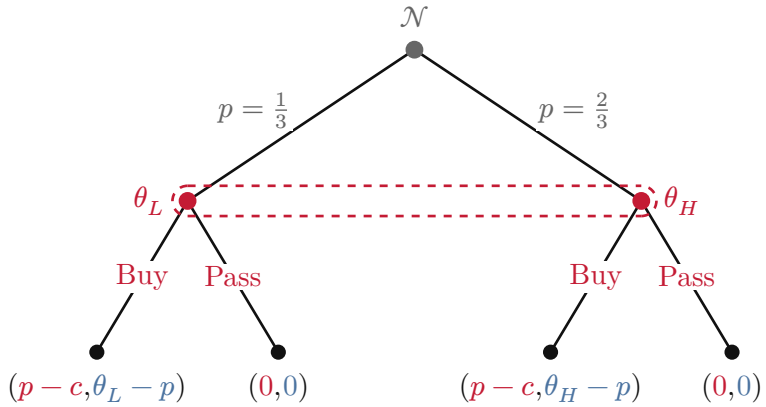

Figure 3: Adverse selection — Nature draws the buyer's type; bracket-style information set on the two buyer nodes.

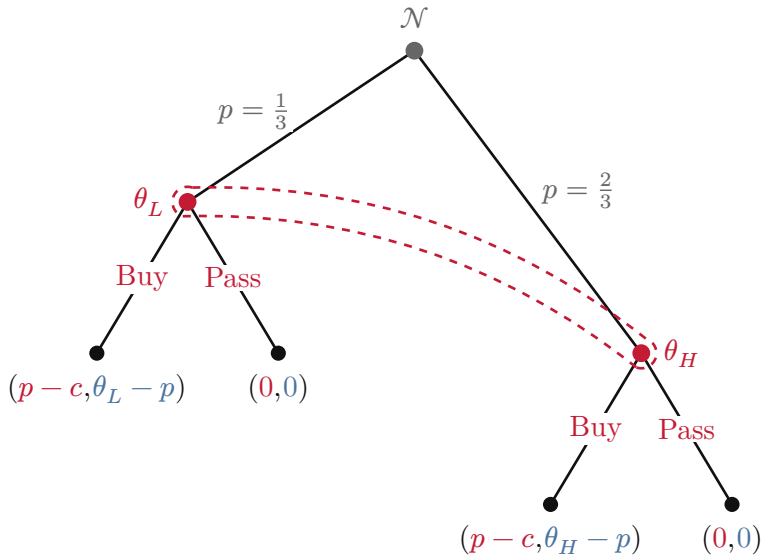

Figure 4: Adverse selection — Nature draws the buyer's type; bracket-style information set on the two buyer nodes.

Example 4 — Beer–Quiche Signalling Game

The classic Cho–Kreps (1987) signalling game. **Nature** draws the **Sender**'s type: *Strong* (S, probability 0.7) or *Weak* (W, probability 0.3). The Sender signals by choosing *Beer* (B) or *Quiche* (Q). The **Receiver** observes the signal but not the type, then chooses *Fight* (F) or *Not Fight* (¬F). Two information sets link nodes reached by the same signal.

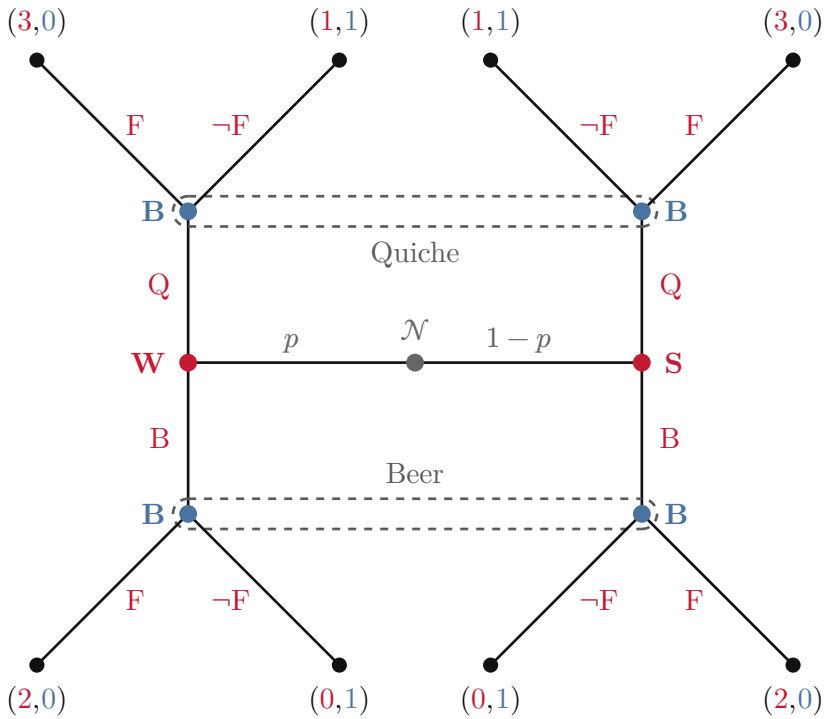

Figure 5: Beer–Quiche signalling game (Cho–Kreps 1987). Payoffs: (Sender, Receiver).

The unique **pooling** Perfect Bayesian Equilibrium has both types choose Beer; the Receiver duels after Quiche (off-path belief: Weak). The PBNE payoffs are (3, 1) for Strong and (2, 0) for Weak.

Macro Reference

Nodes

Macro	Description
`game-node(pos, player:, label:, la:, name:, style:)`	Decision node. `style`: `"filled"` (default), `"open"`, `"dot"`.
`game-nature(pos, label:, la:, name:)`	Nature/chance node (grey, lightly shaded). Default label N.
`game-terminal(pos, payoffs:, la:, parens:)`	Terminal dot + coloured payoff vector.

Branches

Macro	Description
`game-branch(from, to, action:, player:, side:, act:, apos:, arrow:)`	Edge with optional action label. Default: label centred on the line (white background). `side: "e"` / `"w"` offsets the label to the east/west side; `act:` sets the real perpendicular distance (default `game-act`).
`game-prob(from, to, action:, side:, act:, apos:, arrow:)`	Edge with probability label in Nature's grey colour.
`game-highlight(from, to, color:, width:)`	Bold coloured overlay to mark an equilibrium path.

Information Sets & Subgames

Macro	Description
`game-infoset(pos1, pos2, player:, style:, angle:, label:, la:)`	`style: "dashed"` (default) or `"bracket"`. `angle:` bends each segment toward its CCW perpendicular.
`game-subgame(apex, depth:, width:, label:)`	Dotted triangle proper-subgame marker.

Text Helpers

Macro	Description
`game-payoffs(payoffs, parens:)`	Inline coloured payoff vector for body text.
`game-player(player)[label]`	Arbitrary content in that player's colour (bold).
`game-player-default(player)`	Shortcut that produces "Player n" in that player's colour (bold).

Geometry & Style Overrides

To override defaults, redefine the `let` bindings **after** importing:

```
#import "pi-game-trees.typ": *
#let game-R     = 0.18   // larger nodes
#let game-fsl   = 9pt    // bigger player labels
#let game-pal   = (      // custom colours
  rgb("#005f73"), rgb("#94d2bd"), rgb("#e9d8a6"), …
)
```

Coordinate Convention

Trees grow **downward**: root at $y = 0$, leaves at $y < 0$. Typical level spacing: $\Delta y \approx -1.8$ cm; sibling spacing: $\Delta x \approx 1.5$ cm per level of the tree.

For horizontal trees (growing rightward), rotate the coordinate system — set y-differences to 0 and vary x, then use `la: "n"` or `la: "s"` for action labels instead of `"l"` / `"r"`.